Generating and Vaulting a JWT Signing Key for Oracle Integration with OCI Cloud Shell and OCI Vault

Author: Moon Ray Lo, Oracle ACE Apprentice

Date: October 29, 2026

Product: Oracle Cloud Infrastructure Cloud Shell, OCI Vault, Oracle Integration 3

Note: Environment identifiers (OCIDs, client IDs, identity domain and pod identifiers, and email addresses) are masked in the screenshots.

Oracle is retiring password-based authentication for Fusion REST and SOAP services, so I moved my Oracle Integration connections to Fusion onto OAuth using JWT User Assertion. With this policy, Oracle Integration signs a short-lived assertion with a private key, and the identity domain trusts the matching certificate. This contribution shows how I created that key pair without a local OpenSSL install, kept the private key out of my laptop, and stored a recoverable copy in OCI Vault.

Part 1: Generate the key pair and certificate in Cloud Shell

I opened OCI Cloud Shell from the console. Cloud Shell already has OpenSSL and the OCI CLI, and its session runs in the tenancy home region regardless of the region selected in the console. I set umask 077 so every file I created would be readable only by me, then created a working folder.

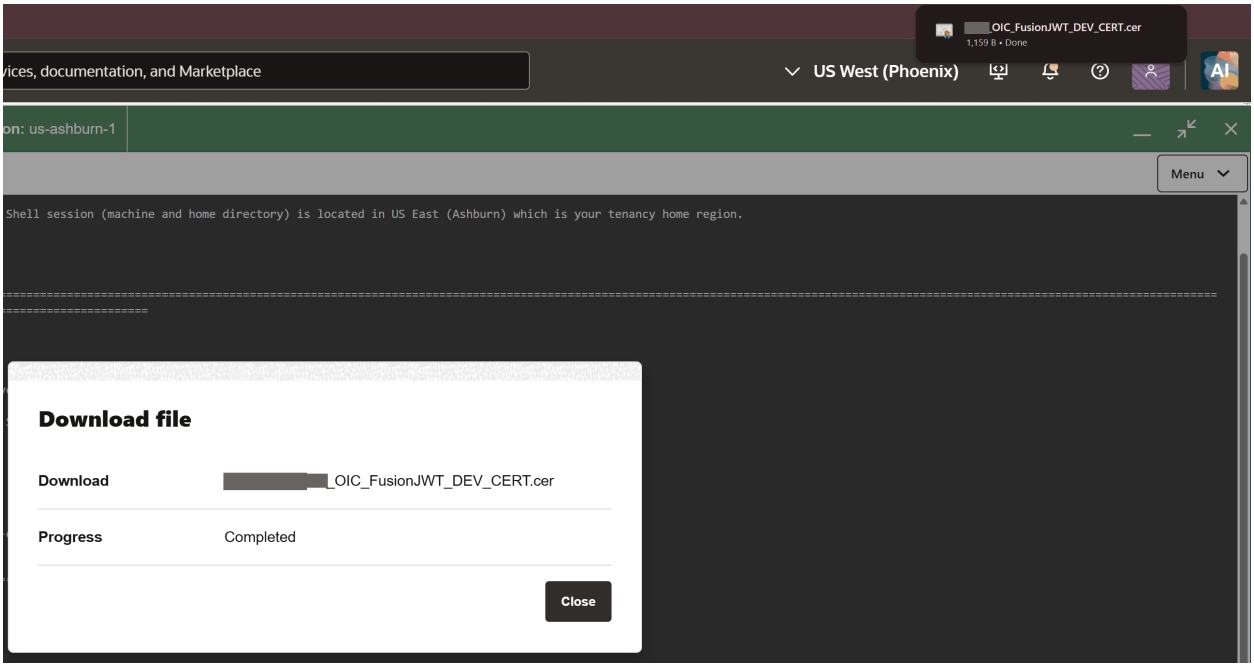
I generated a 2048-bit RSA key and a self-signed certificate valid for one year, using the certificate alias as the common name. I then printed the subject, end date and SHA-256 fingerprint, and compared the modulus of the certificate with the modulus of the private key to confirm they belong together.

```
mraylo@cloudshell:~ (us-phoenix-1)$ umask 077
mraylo@cloudshell:~ (us-phoenix-1)$ mkdir -p ~/[redacted]-oauth && cd ~/[redacted]-oauth
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ ALIAS=[redacted]_OIC_FusionJWT_DEV_CERT
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
>   -keyout ${ALIAS}_private_key.pem -out ${ALIAS}.cer -subj "/CN=${ALIAS}"
Generating a RSA private key
..........................................................................................................+++++
+++++
writing new private key to '[redacted]_OIC_FusionJWT_DEV_CERT_private_key.pem'
-----
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ cd ~/[redacted]-oauth
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ ALIAS=[redacted]_OIC_FusionJWT_DEV_CERT
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ openssl x509 -in ${ALIAS}.cer -noout -subject -enddate -fingerprint -sha256
subject=CN = [redacted]_OIC_FusionJWT_DEV_CERT
notAfter=Oct 29 04:55:27 2027 GMT
SHA256 Fingerprint=[redacted]
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ head -1 ${ALIAS}_private_key.pem
-----BEGIN PRIVATE KEY-----
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ [ "$(openssl x509 -in ${ALIAS}.cer -noout -modulus)" = \
>   "$(openssl rsa -in ${ALIAS}_private_key.pem -noout -modulus)" ] && echo "Key and certificate match"
Key and certificate match
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$ ls -l
total 8
-rw-------. 1 mraylo oci 1159 Oct 29 04:55 [redacted]_OIC_FusionJWT_DEV_CERT.cer
-rw-------. 1 mraylo oci 1704 Oct 29 04:55 [redacted]_OIC_FusionJWT_DEV_CERT_private_key.pem
mraylo@cloudshell:[redacted]-oauth (us-phoenix-1)$
```

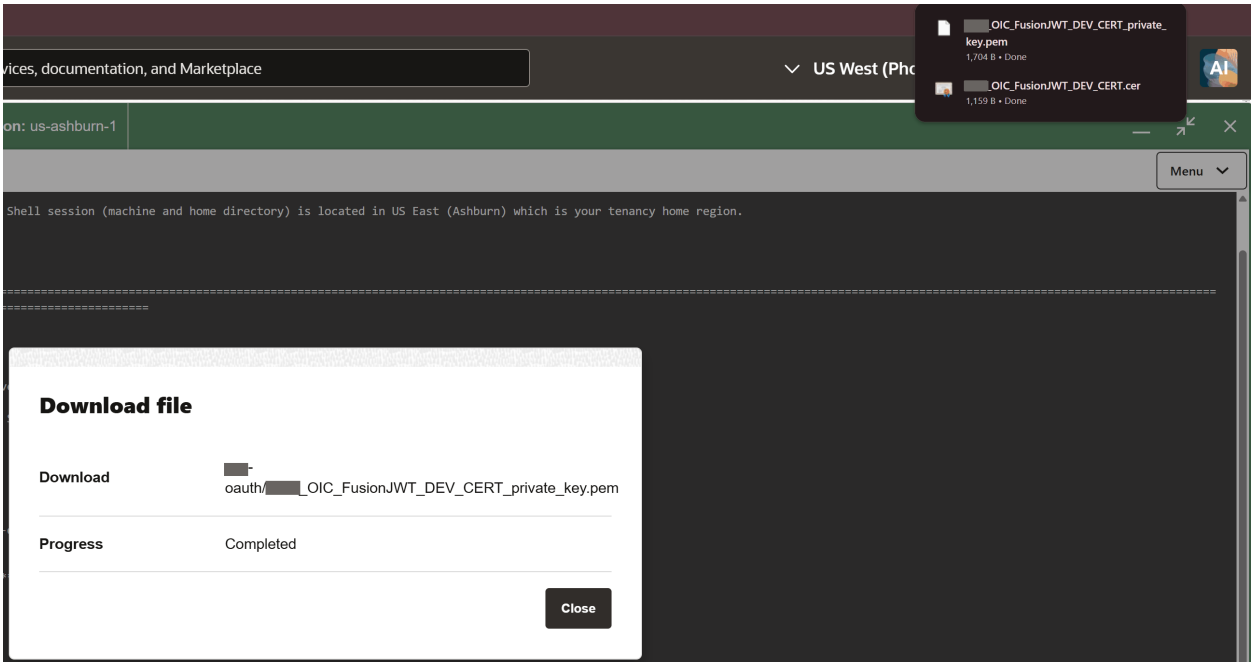
OpenSSL in Cloud Shell: key pair and certificate generated, expiry and fingerprint printed, and the key and certificate confirmed as a matching pair.

Part 2: Download the certificate and the private key

I used Menu > Download in Cloud Shell to download both files. The path is entered relative to the home folder. The certificate (.cer) is imported into the identity domain later, and the private key (.pem) is uploaded to Oracle Integration. Once both uploads were done, I deleted the private key from my Downloads folder.



Cloud Shell download of the certificate.

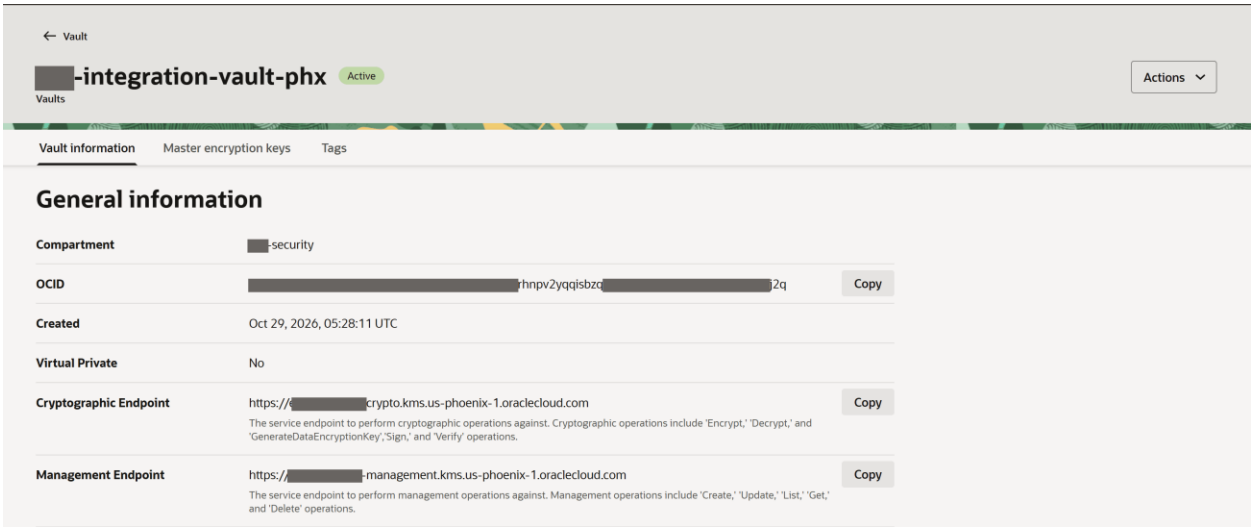


Cloud Shell download of the private key.

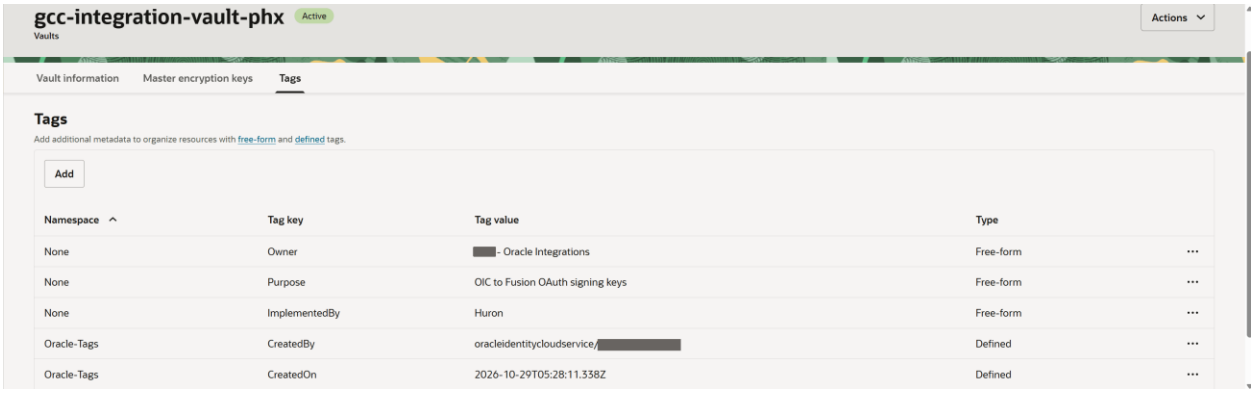
Part 3: Create the vault, master encryption key and tags

Keeping the only copy of the private key on a laptop or in Cloud Shell is risky, so I created a dedicated OCI Vault in a security compartment rather than the root compartment. I chose a virtual vault (not virtual private) and a software-protected AES 256 master encryption key. This combination, together with the secret, has no cost.

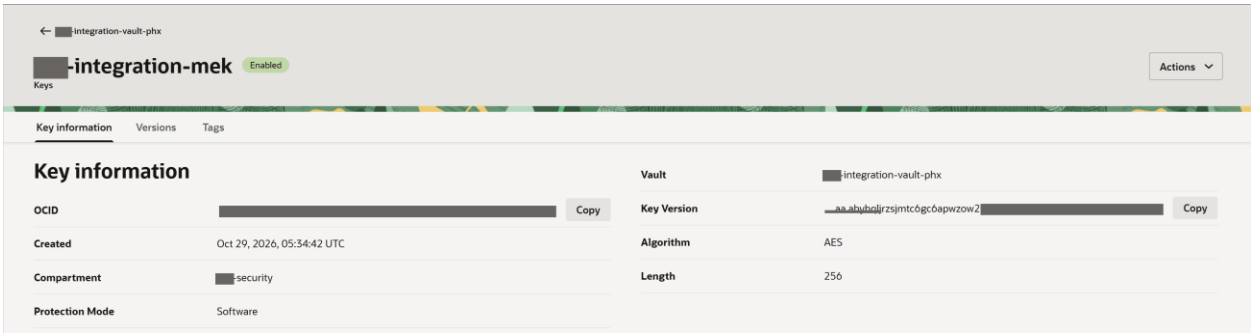
I tagged the vault with Owner, Purpose and ImplementedBy. The Owner tag names the team that will own the integration after go-live, because the certificate renewal will happen long after the build team has moved on.



The new vault, Active, created as a virtual vault.



Free-form tags for Owner, Purpose and ImplementedBy alongside the Oracle default tags.



The master encryption key: software protection, AES, 256-bit, Enabled.

Part 4: Store the private key as a secret and verify it

Back in Cloud Shell, I looked up the vault compartment and created the secret with oci vault secret create-base64, passing the private key as base64 content and the same tags as the vault. To prove the stored copy was usable, I read the secret bundle back, decoded it, and hashed the RSA modulus with sha256sum. The hash matched the original key.

With a verified copy in the vault, I shredded the working copy of the private key in Cloud Shell and kept only the certificate.

```
mraylo@cloudshell:    -oauth (us-phoenix-1)$ KEY=
mraylo@cloudshell:    -oauth (us-phoenix-1)$ COMP=$(oci kms management vault get --vault-id $VAULT --query 'data."compartment-id"' --raw-output)
mraylo@cloudshell:    -oauth (us-phoenix-1)$ echo "Compartment: $COMP"
Compartment:
mraylo@cloudshell:    -oauth (us-phoenix-1)$ SECRET=$(oci vault secret create-base64 \
>   --compartment-id $COMP --vault-id $VAULT --key-id $KEY \
>   --secret-name ${ALIAS}_PRIVATE_KEY \
>   --description "OIC DEV JWT signing key for Fusion DEV1; expires 2027-10-29" \
>   --freeform-tags '{"Owner":"    - Oracle Integrations","Purpose":"OIC to Fusion OAuth signing keys","ImplementedBy":"Huron"}' \
>   --secret-content-content "$(base64 -w0 ${ALIAS}_private_key.pem)" \
>   --query 'data.id' --raw-output)
mraylo@cloudshell:    -oauth (us-phoenix-1)$ echo "Secret: $SECRET"
Secret:
mraylo@cloudshell:    -oauth (us-phoenix-1)$ oci secrets secret-bundle get --secret-id $SECRET \
>   --query 'data."secret-bundle-content".content' --raw-output | base64 -d | \
>   openssl rsa -noout -modulus | sha256sum
                                                                                -
```

Secret created in the vault, then read back and verified by hashing the key modulus.

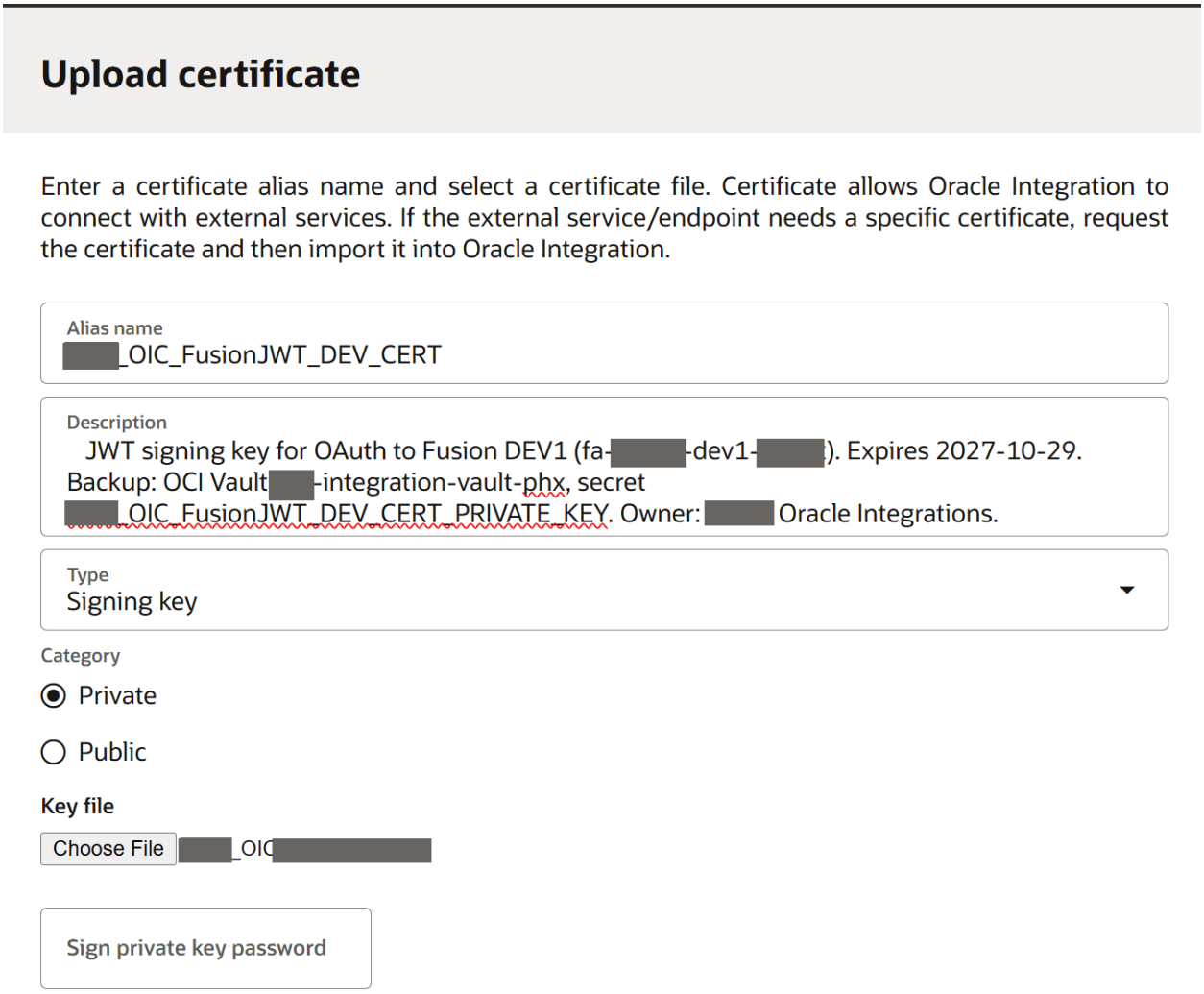
```
mraylo@cloudshell:    -oauth (us-phoenix-1)$ echo "Secret: $SECRET"
Secret:
mraylo@cloudshell:    -oauth (us-phoenix-1)$ oci secrets secret-bundle get --secret-id $SECRET \
>   --query 'data."secret-bundle-content".content' --raw-output | base64 -d | \
>   openssl rsa -noout -modulus | sha256sum
                                                                                -
mraylo@cloudshell:    -oauth (us-phoenix-1)$ shred -u ~/    -oauth/    _OIC_FusionJWT_DEV_CERT_private_key.pem && ls -l ~/    -oauth
total 4
-rw-------. 1 mraylo oci 1159 Oct 29 04:55     _OIC_FusionJWT_DEV_CERT.cer
mraylo@cloudshell:    -oauth (us-phoenix-1)$
```

The working copy of the private key shredded; only the certificate remains in the folder.

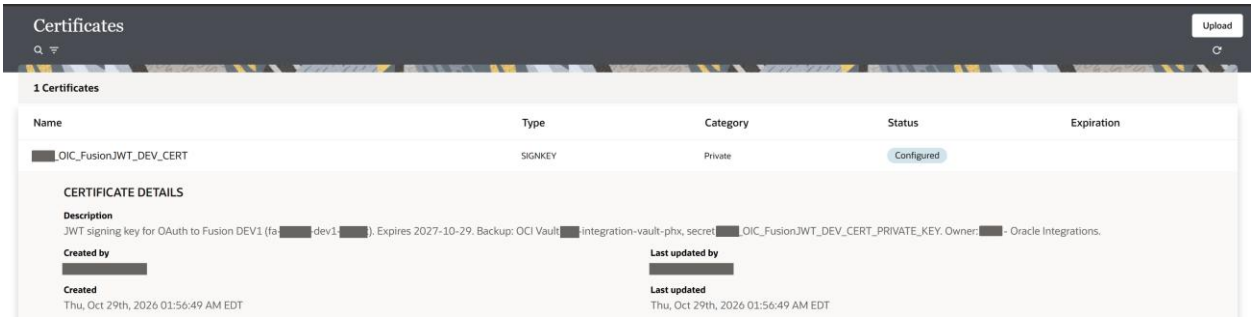
Part 5: Upload the signing key to Oracle Integration

In Oracle Integration, I went to Settings > Certificates > Upload and chose Type Signing key and Category Private. I used the same alias as the certificate common name, because the same value must appear as the kid in the JWT header and as the certificate alias in the identity domain. In the description I recorded the expiry date, the vault and secret name, and the owner.

The certificate list shows the entry as SIGNKEY, Private, Configured. Oracle Integration does not show an expiration date for signing keys, which is why I recorded the expiry in the description and in a certificate register.



Uploading the private key to Oracle Integration as a private signing key.



The signing key listed in Oracle Integration with its description.

What I learned

Cloud Shell made key generation simple and kept the private key off my machine, but I had to remember that it runs in the home region. The modulus-hash check gave me confidence that the vaulted key can actually rebuild the connection if Oracle Integration is ever recreated. Recording the expiry outside Oracle Integration is essential, because the certificate list will not warn you.